



Universidad Autónoma de Baja California

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA



Microprocesadores y Microcontroladores.

Práctica No. 1: Operadores Bitwise y Fundamentos de C para Sistemas Embebidos.

Alumno:

Joshua Osorio O. – 1293271

Docente:

Jesús A. García L.

Agosto/2026

Tabla de contenido

I. Objetivo	4
II. Descripción de la practica.....	4
Lista de materiales.....	4
III. Desarrollo de la práctica	4
Teoría	4
Lenguaje C en sistemas embebidos	4
Compilador cruzado (cross-compiler)	5
Tamaño de las primitivas y uso de inttypes.h.....	5
Representación de literales.....	5
Conceptos investigados	5
Operadores bitwise.....	5
Apuntadores	6
Estructuras (struct, union, bitfield)	6
Palabras clave: const, static y volatile.....	6
Condicionales del pre-compilador.....	6
Casteo (type casting).....	7
Control de versiones con git	7
Ejercicios resueltos (enmascaramiento, apuntadores y estructuras)	7
Ejercicio 1 — objetivo 0xB0B0.....	7
Ejercicio 2 — objetivo 0xBABA.....	7
Ejercicio 3 — objetivo 0xBEBE	7
Ejercicio 4 — objetivo 0xFE00	7
Ejercicio 5 — objetivo 0xFEAA	8
Ejercicio 6 — objetivo 0x0155	8
Ejercicio 7 — objetivo 0x0001	8
Ejercicio 8 — objetivo 0x0100	8
Ejercicio 9 — apuntadores, objetivo 0xDEADBEEF.....	8
Ejercicio 10 — estructuras, objetivo 0xBADDC0DE.....	8
Ejercicio 11 — bitfiddling, empaçar un arreglo en 32 bits.....	9
Resultados de ejecución	9
Reflexión y análisis crítico	9

Dificultades técnicas..... 9

Fracasos, fallas, bugs y errores..... 9

Cómo se resolvió..... 10

Conclusión 10

IV. Bibliografía..... 10

I. Objetivo

El alumno se familiarizará con usos típicos del lenguaje C dentro de sistemas embebidos: el manejo de tipos de datos de tamaño fijo y la librería `inttypes.h`, la representación de literales en distintas bases, los operadores a nivel de bits (bitwise), el uso de apuntadores y de estructuras con campos de bits, así como los conceptos de compilador cruzado, palabras clave de calificación (`const`, `static`, `volatile`), condicionales del pre-compilador, casteo de tipos y control de versiones con git.

II. Descripción de la practica

Investigar el tamaño de las primitivas de C y el uso de la librería `inttypes.h`.

Investigar la representación de literales en distintas bases (decimal, hexadecimal `0x1`, binario `0b1`, octal `0`).

Investigar los operadores bitwise (`&`, `|`, `~`, `^`, `>>`, `<<`) y diferenciarlos de los operadores lógicos (`&&`, `||`, `!`).

Investigar el uso de apuntadores (`char *x`, `int (*cb)(void)`).

Investigar las estructuras (`struct`, `union`, `bitfield`).

Investigar las palabras clave `const`, `static` y `volatile`.

Investigar los condicionales del pre-compilador (`#`).

Investigar el casteo (type casting).

Investigar el uso básico de git (GitHub).

Realizar los ejercicios de la práctica relacionados con enmascaramiento de bits, apuntadores y estructuras.

Redactar las conclusiones y comentarios de la práctica.

Subir el código a un repositorio y el reporte en el apartado de Taller.

Lista de materiales

- Computadora personal (PC).
- Compilador de C / entorno de desarrollo cruzado (Atmel Studio, AVR-GCC o GCC estándar para pruebas en escritorio).
- Editor de texto y terminal con acceso a git.

III. Desarrollo de la práctica

Teoría

Lenguaje C en sistemas embebidos

El lenguaje C sigue siendo el estándar de facto para programar sistemas embebidos porque ofrece control directo sobre la memoria y el hardware (apuntadores, acceso a direcciones específicas, control de tamaño de tipos) sin dejar de ser portable entre arquitecturas, y su código se traduce a instrucciones de máquina muy eficientes, algo crítico en microcontroladores con poca memoria de programa y RAM. Por ello se usa para escribir desde el arranque del sistema (bootloaders) hasta controladores de periféricos, temporizadores e interrupciones.

Compilador cruzado (cross-compiler)

Un compilador cruzado (cross-compiler) es un compilador que se ejecuta en una computadora con una arquitectura (por ejemplo, x86/x64 de una PC) pero genera código ejecutable para una arquitectura distinta, la del microcontrolador de destino (por ejemplo, AVR de 8 bits). Esto es necesario porque el microcontrolador normalmente no tiene la capacidad de compilar su propio software: se programa y compila en la PC y después se transfiere (flashea) el binario resultante a la memoria de programa del microcontrolador mediante un programador o bootloader.

Tamaño de las primitivas y uso de inttypes.h

El tamaño en bits de los tipos básicos de C (char, int, long, etc.) no está fijado por el estándar, sino que depende del compilador y de la arquitectura de destino; en un microcontrolador de 8 bits un int puede ocupar 16 bits, mientras que en una PC de escritorio suele ocupar 32. Esta ambigüedad es riesgosa en sistemas embebidos, donde el tamaño exacto de una variable determina si cabe en un registro de hardware o si desborda. Por eso la librería estándar <inttypes.h> (que incluye a <stdint.h>) define tipos de ancho fijo como uint8_t, int8_t, uint16_t, int16_t, uint32_t e int32_t, cuyo tamaño en bits es siempre el mismo sin importar el compilador o la plataforma, además de macros de formato para printf/scanf como PRIu32 o PRIx16.

```
#include <inttypes.h>

uint8_t a; /* entero sin signo de 8 bits: 0 a 255 */
uint16_t b; /* entero sin signo de 16 bits: 0 a 65535 */
uint32_t c; /* entero sin signo de 32 bits */

printf("sizeof(uint16_t) = %zu bytes\n", sizeof(uint16_t));
printf("valor de b en hex = %" PRIx16 "\n", b);
```

Representación de literales

En C, un mismo valor entero puede escribirse en distintas bases usando prefijos: sin prefijo se interpreta en decimal (por ejemplo 1); con el prefijo 0x (o 0X) se interpreta en hexadecimal (0x1); con el prefijo 0b (o 0B, extensión soportada por GCC y por C23) se interpreta en binario (0b1); y un literal que comienza únicamente con 0 se interpreta en octal, de modo que 0 y 010 no valen lo mismo que en decimal (010 en octal equivale a 8 en decimal). El compilador convierte cualquiera de estas representaciones al mismo valor binario interno; la base elegida es solo una conveniencia de lectura para el programador, y en programación de bajo nivel se prefiere hexadecimal o binario porque reflejan directamente los bits que se están manipulando.

```
int decimal = 10; /* base 10 */
int hexa    = 0xA; /* base 16, mismo valor: 10 */
int binario = 0b1010; /* base 2, mismo valor: 10 */
int octal   = 012; /* base 8, mismo valor: 10 */
```

Conceptos investigados

Operadores bitwise

Los operadores bitwise (&, |, ~, ^, >>, <<{) trabajan sobre la representación binaria de un valor, bit por bit, y son la base de las técnicas de enmascaramiento: AND (&) se usa para apagar bits o para leer bits específicos,

OR (|) para encender bits, XOR (^) para invertir (toggle) solo los bits que están en 1 en la máscara, NOT (~) para invertir todos los bits, y los corrimientos (<<, >>) para reubicar un grupo de bits a otra posición. Son operadores completamente distintos de los operadores lógicos ({&&, ||, !}), que trabajan sobre el valor completo como una condición booleana (verdadero/falso) y se usan para combinar condiciones en un if, no para modificar bits individuales; confundir & con && (o | con ||) es un error común que compila sin errores pero produce un resultado incorrecto.

Apuntadores

Un apuntador es una variable que guarda la dirección de memoria de otro dato en lugar del dato mismo. La declaración `char *x` define un apuntador a un `char`: `x` guarda una dirección, y `*x` accede (dereferencia) al byte almacenado en esa dirección. Los apuntadores permiten reinterpretar una misma región de memoria con distintos tipos, lo cual se usa mucho en sistemas embebidos para acceder a registros de hardware en direcciones fijas. También existen apuntadores a función, como en `int (*cb)(void)`, que guardan la dirección de una función en memoria de programa en lugar de la de un dato, y se usan para implementar callbacks o tablas de manejadores de interrupción.

Estructuras (struct, union, bitfield)

Una struct agrupa varias variables (miembros), posiblemente de tipos distintos, bajo un mismo nombre, reservando memoria para todos sus miembros de forma consecutiva. Una union, en cambio, reserva memoria para un solo miembro a la vez: todos sus miembros comparten la misma dirección, y el tamaño de la unión es el de su miembro más grande, lo que sirve para reinterpretar el mismo bloque de memoria de formas distintas. Un bitfield es un miembro de una struct o union al que se le especifica un número exacto de bits (por ejemplo, `unsigned flag : 1;`), lo que permite empaquetar varias banderas o campos pequeños dentro de la misma palabra de memoria, algo muy usado para mapear registros de hardware bit a bit.

Palabras clave: const, static y volatile

`const` indica que una variable no debe modificarse después de inicializarse; el compilador genera un error si el código intenta reasignarla, lo cual ayuda a documentar la intención y a evitar modificaciones accidentales. `static`, aplicada a una variable local, hace que conserve su valor entre llamadas a la función (se inicializa una sola vez y vive durante todo el programa); aplicada a una variable o función a nivel de archivo, limita su visibilidad (linkage) a ese mismo archivo. `volatile` le indica al compilador que el valor de esa variable puede cambiar en cualquier momento por causas externas al flujo normal del programa (una interrupción, un periférico de hardware), por lo que no debe optimizar el acceso a ella asumiendo que su valor permanece igual entre lecturas.

Condicionales del pre-compilador

Los condicionales del pre-compilador (`#if`, `#ifdef`, `#ifndef`, `#else`, `#elif`, `#endif`, junto con `#define`) se evalúan antes de que el código llegue al compilador propiamente dicho, sobre el texto fuente. Se usan para incluir o excluir bloques de código según constantes definidas en tiempo de compilación, por ejemplo para compilar una misma base de código hacia distintos microcontroladores (`#ifdef __AVR_ATmega328P__`), para evitar inclusiones múltiples de un mismo encabezado (include guards), o para activar código de depuración solo cuando se define `DEBUG`, sin que ese código exista siquiera en la versión final compilada.

Casteo (type casting)

El casteo (type casting) es la conversión explícita del valor de un tipo de dato a otro, escribiendo el tipo destino entre paréntesis antes de la expresión, por ejemplo `(uint16_t)valor` o `(uint8_t *)&b`. C también realiza conversiones implícitas de forma automática en ciertas operaciones (por ejemplo, al mezclar `int` y `double` en una misma expresión), pero un casteo explícito documenta la intención del programador y es indispensable al reinterpretar apuntadores, como al convertir la dirección de una variable `uint32_t` en un apuntador a `uint16_t` para acceder a sus dos mitades por separado.

Control de versiones con git

git es un sistema de control de versiones distribuido que registra el historial de cambios de un proyecto mediante commits, permite trabajar en ramas (branches) independientes y facilita volver a versiones anteriores del código o comparar diferencias entre ellas. GitHub es un servicio en línea que aloja repositorios de git y añade herramientas de colaboración (pull requests, issues, revisión de código). El flujo básico de trabajo consiste en clonar o inicializar un repositorio, hacer cambios locales, agregarlos al área de preparación (git add), confirmarlos con un mensaje descriptivo (git commit) y subirlos al repositorio remoto (git push).

Ejercicios resueltos (enmascaramiento, apuntadores y estructuras)

En todos los ejercicios se parte de `a = 0x0000` (o `b = 0x1155` en los de apuntadores/estructuras) y se transforma únicamente con operadores bitwise hasta llegar al valor objetivo indicado en cada comentario del código fuente (Prac1.c), verificado con la macro `TEST_ASSERT`.

Ejercicio 1 — objetivo 0xB0B0

```
a |= (1 << 15) | (3 << 12) | (1 << 7) | (3 << 4);
```

Partiendo de `a = 0x0000`, todos los bits que difieren respecto al valor buscado (`B0B0 = 1011 0000 1011 0000`) deben pasar de 0 a 1, así que basta un solo OR con una máscara que enciende exactamente esas posiciones.

Ejercicio 2 — objetivo 0xBABA

```
a |= (1 << 15) | (3 << 12) | (1 << 11) | (1 << 9) | (1 << 7) | (3 << 4) | (1 << 3) | (1 << 1);
```

De nuevo se parte de `0x0000`, así que todos los bits en 1 de `BABA` deben encenderse; se resuelve con un único OR.

Ejercicio 3 — objetivo 0xBEBE

```
a |= (1 << 10) | (1 << 2);
```

Comparando `BABA (1011 1010 1011 1010)` con `BEBE (1011 1110 1011 1110)` solo cambian dos bits, y ambos pasan de 0 a 1, por lo que se aplica OR únicamente sobre esas dos posiciones.

Ejercicio 4 — objetivo 0xFE00

```
a |= (1 << 14);
```

```
a &= ~( (1 << 7) | (3 << 4) | (1 << 3) | (3 << 1) );
```

Aquí hay bits que deben encenderse (bit 14) y otros que deben apagarse (los bits bajos de BEBE), así que se combina OR para encender y AND con la máscara negada (~) para apagar.

Ejercicio 5 — objetivo 0xFEAA

```
a |= (1 << 7) | (1 << 5) | (1 << 3) | (1 << 1);
```

Todos los cambios respecto a FE00 son de 0 a 1 (se encienden los bits de la mitad baja), por lo que vuelve a bastar un OR.

Ejercicio 6 — objetivo 0x0155

```
a = ~a;
```

0155 (0000 0001 0101 0101) es exactamente el complemento bit a bit de FEAA (1111 1110 1010 1010); la forma más directa de obtenerlo es negar todos los bits con el operador ~.

Ejercicio 7 — objetivo 0x0001

```
a &= ~( (1 << 8) | (1 << 6) | (1 << 4) | (1 << 2) );
```

Todos los cambios respecto a 0155 apagan bits, así que se usa AND con la máscara negada de los bits que deben quedar en 0.

Ejercicio 8 — objetivo 0x0100

```
a |= (1 << 8);
```

```
a &= ~(1 << 0);
```

Se necesita encender el bit 8 y apagar el bit 0 al mismo tiempo, por lo que se combinan OR y AND con máscara negada, igual que en el ejercicio 4.

Ejercicio 9 — apuntadores, objetivo 0xDEADBEEF

```
uint32_t b = 0x1155;
b = ((~b) << 16) | b; /* b = 0xEEAA1155 */
uint16_t *ptr16 = (uint16_t *)&b;
*(ptr16 + 1) ^= (3 << 12) | 7;
*ptr16 ^= (10 << 12) | (15 << 8) | (11 << 4) | 10;
```

Se reinterpreta la dirección de 'b' como un arreglo de dos uint16_t mediante un apuntador: ptr16[1] direcciona la mitad alta (EEAA) y ptr16[0] la mitad baja (1155). Como en ambas mitades hay bits que deben apagarse y otros encenderse a la vez, se usa XOR con una máscara que invierte únicamente los bits que difieren entre el valor actual y el buscado (DEAD y BEEF respectivamente).

Ejercicio 10 — estructuras, objetivo 0xBADDC0DE

```
typedef struct { uint16_t bl; uint16_t bh; } EBX;
EBX *ptr = (EBX *)&b;
ptr->bh ^= (6 << 12) | (4 << 8) | (7 << 4) | 0;
ptr->bl ^= (7 << 12) | (14 << 8) | (3 << 4) | 1;
```

Se mapea la misma memoria de 'b' con una estructura de dos campos de 16 bits (bl y bh) para acceder a cada mitad por nombre en lugar de aritmética de apuntadores, aplicando la misma técnica de XOR selectivo para pasar de DEADBEEF a BADDC0DE.

Ejercicio 11 — bitfiddling, empaquetar un arreglo en 32 bits

```
uint8_t arreglo[] = {0x18, 0x13, 0x15, 0x08, 0x03, 0x09};
b = 0;
for (int i = 0; i < 6; i++) {
    b |= ((uint32_t)(arreglo[i] & 0x1F)) << (i * 5);
}
```

Cada elemento del arreglo se enmascara con 0x1F para conservar solo sus 5 bits menos significativos y se recorre (<<) a su posición correspondiente dentro de 'b' según el índice i, acumulando todo con OR; el resultado es b = 0x12345678 en formato little-endian, con los 3 bits más altos en 0.

Resultados de ejecución

El código completo se compiló sin errores con gcc -Wall (una única advertencia inofensiva por la variable ptr8, declarada para ejemplificar el acceso por bytes pero no usada en la versión final) y al ejecutarse produjo el siguiente resultado, confirmando que los 10 ejercicios verificados con TEST_ASSERT pasan correctamente y que el ejercicio 11 produce el valor esperado 0x12345678:

```
a = B0B0 [PASS] Ejercicio 1
a = BABA [PASS] Ejercicio 2
a = BEBE [PASS] Ejercicio 3
a = FE00 [PASS] Ejercicio 4
a = FEAA [PASS] Ejercicio 5
a = 155 [PASS] Ejercicio 6
a = 1 [PASS] Ejercicio 7
a = 100 [PASS] Ejercicio 8
b = EEAA1155
b = DEADBEEF [PASS] Ejercicio 9
b = BADDC0DE [PASS] Ejercicio 10
b = 12345678
```

Reflexión y análisis crítico

Dificultades técnicas

La principal dificultad fue decidir, para cada transición de valores, qué operador bitwise correspondía usar: cuando todos los bits cambiantes pasaban de 0 a 1 bastaba con OR, cuando pasaban de 1 a 0 se necesitaba AND con la máscara negada (~), y cuando había bits que se encendían y otros que se apagaban simultáneamente había que combinar ambos, o bien usar XOR cuando el patrón de cambio coincidía exactamente con una máscara de diferencia. Otra dificultad fue razonar el orden de los bytes (little-endian) al reinterpretar 'b' a través de apuntadores de 16 bits y de la estructura de campos de bits, ya que la mitad que aparentemente es la 'primera' en memoria corresponde a la parte baja del valor, no a la alta.

Fracasos, fallas, bugs y errores

En los intentos previos a la solución final, el error más frecuente fue de ubicación: al construir las máscaras no siempre se identificaba correctamente en qué posición (bit) debía aplicarse el corrimiento, es decir, cuántos lugares había que recorrer un 1 (o un 0) para que cayera exactamente sobre el bit que se quería encender o apagar, lo que llevaba a máscaras que activaban bits equivocados y a que las macros TEST_ASSERT

fallaran mostrando un valor distinto al esperado. Este problema se presentó sobre todo en los ejercicios de enmascaramiento con más bits dispersos, como al transformar hacia BABA, y también en los ejercicios de apuntadores al buscar DEADBEEF. Por otro lado, en el ejercicio 11 (recorrer el arreglo y extraer únicamente los 5 bits menos significativos de cada elemento para empacarlos en 'b') el reto no fue de sintaxis sino de lógica: no era evidente a simple vista por qué había que desplazar cada grupo de 5 bits una cantidad distinta según la posición del elemento en el arreglo.

Cómo se resolvió

Para los errores de ubicación de bits se adoptó la misma estructura de comentario que el profesor propone como ejemplo (mostrar el valor en binario alineado bit a bit) y se le añadió una regla de índices debajo de cada valor (5432 1098 7654 3210) para poder ubicar de un vistazo cuántas posiciones había que recorrer un 1 o un 0 antes de armar la máscara; esta misma técnica se reutilizó tanto en el ejercicio de BABA como en el de DEADBEEF, y permitió corregir las máscaras comparando directamente el bit por bit "tengo" contra "busco". Para el ejercicio 11, en cambio, la solución fue hacer el desglose a mano en el cuaderno: escribir cada elemento del arreglo en binario, marcar sus 5 bits menos significativos y calcular a mano cuántos lugares debía recorrerse cada grupo según su índice dentro del arreglo, hasta entender el patrón ($i * 5$). Con esa lógica ya clara, se programó primero una versión paso a paso usando variables temporales (tmp, tmp2) para verificar cada etapa por separado (enmascarar, convertir de tipo, recorrer y acumular), y una vez confirmado que esa versión daba el resultado correcto, se condensó todo en una sola línea dentro del ciclo for.

Conclusión

Esta práctica permitió reforzar de forma muy concreta el manejo de operadores bitwise para leer, encender, apagar e invertir bits individuales, así como el uso de tipos de tamaño fijo (uint8_t, uint16_t, uint32_t) que son indispensables al trabajar con registros de hardware en sistemas embebidos. Además, resolver los ejercicios de apuntadores y de estructuras con campos de bits sobre la misma variable 'b' ayudó a entender que una región de memoria puede reinterpretarse de varias formas (como bytes, como medias palabras o como campos con nombre), lo cual es una técnica común al mapear periféricos de un microcontrolador.

IV. Bibliografía

- [1] Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall.
- [2] ISO/IEC. (2018). ISO/IEC 9899:2018 — Information technology — Programming languages — C.
- [3] Microchip Technology / Atmel. (2016). ATmega48A/PA/88A/PA/168A/PA/328/P Datasheet (DS40002061B).
- [4] Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. Recuperado de <https://git-scm.com/book>